

Mini Project — Part D: Scenario-Based Assessment

UST — 100 Next Gen AI Engineer Cohort 3

Six business cases across Finance, Telecom and Healthcare. The focus is on deciding **when to use ML, DL, GenAI or RAG**, and on problem formulation, technology selection, architecture, model selection, evaluation and business implications.

Scenario 1 — Finance: Credit Default Prediction

A bank has 2 million customer records (income, credit score, loan amount, employment history, previous defaults, transaction behaviour, debt-to-income ratio) and wants to predict whether a new borrower will default.

1. Would you use ML or DL?

Machine Learning — specifically gradient-boosted trees. The data is structured and tabular, and on tabular data gradient boosting (XGBoost / LightGBM) consistently matches or beats neural networks while training far faster and needing much less tuning. 2 million rows sounds like “big data,” but volume alone does not justify deep learning; what matters is that the features are already well-engineered numeric/categorical columns, which is exactly where tree ensembles excel. There is also a hard practical constraint: credit decisions must be explainable to regulators and to the customer being declined, and a boosted tree with SHAP values is far easier to defend than a neural network.

2. Which algorithms would you consider?

- **Logistic Regression** — the baseline, and still the industry standard for regulatory scorecards because coefficients translate directly into points-based scoring and are trivially explainable.
- **Random Forest** — a robust non-linear benchmark.
- **XGBoost / LightGBM** — the likely best performer; LightGBM in particular handles 2M rows efficiently.
- **CatBoost** — worth testing if there are high-cardinality categoricals (employer, region).

I would run Logistic Regression as the interpretable benchmark and a boosted model as the challenger, then decide based on the accuracy gain versus the explainability cost.

3. What should the target variable be?

A binary default flag, but it must be defined with an explicit **performance window** rather than left vague. For example: default = 1 if the borrower is 90+ days past due (DPD 90+) within 12 months of loan origination. Three details matter: the delinquency threshold (90 DPD is the common regulatory definition), the observation window (12 months), and the exclusions (accounts closed early, fraud cases, or accounts too new to have completed the window). Getting this definition wrong is a bigger risk to the project than the choice of algorithm.

4. Which evaluation metrics would you prioritise?

- **PR-AUC** (precision-recall AUC) as the headline metric, because defaults are a small minority class and PR-AUC is far more informative than accuracy or even ROC-AUC under imbalance.
- **Recall at a fixed precision / approval rate** — the operationally meaningful version: “of all borrowers who will default, what share do we catch, at a decline rate the business can tolerate?”

- **KS statistic and Gini** — the standard discrimination measures in credit risk, expected by risk teams and regulators.
- **Calibration** (calibration curve, Brier score) — critical and often forgotten. The predicted probability of default feeds directly into pricing and expected-loss/provisioning calculations, so a model that ranks well but outputs mis-scaled probabilities is unusable for those purposes.

5. Why might recall be more important than accuracy?

Because the classes are imbalanced and the costs are asymmetric. If only 3% of borrowers default, a model that predicts “no default” for everyone achieves 97% accuracy while catching zero defaults — a perfect accuracy score and a completely worthless model.

The cost asymmetry is the real argument. A **false negative** is a borrower we approved who then defaults: the bank can lose most of the principal, often many thousands. A **false positive** is a good borrower we declined: the bank loses the interest margin on one loan and some goodwill. The FN is typically an order of magnitude more expensive, so we deliberately favour recall — accepting more false positives to miss fewer defaults. In practice the decision threshold should be set from an explicit cost matrix (expected loss given default versus lost margin), not left at the default 0.5.

6. How would you handle class imbalance?

In order of preference:

- **Cost-sensitive learning** — `scale_pos_weight` in XGBoost or `class_weight='balanced'`, which tells the model that a missed default matters more than a false alarm. This is my first choice because it changes the objective without fabricating data.
- **Threshold tuning** — keep the model as-is and move the decision threshold to the point that optimises expected cost. Simple, transparent, and easy to justify to a regulator.
- **Stratified sampling** in all train/test splits and cross-validation folds, so the class ratio is preserved.
- **Metric choice** — evaluate on PR-AUC and recall rather than accuracy, so imbalance cannot hide in the score.
- **SMOTE / synthetic oversampling** — possible, but I would use it cautiously and only ever fit it on the training folds. In credit risk, synthetic borrowers are hard to defend (“this applicant profile does not exist”) and can distort calibration, so cost weighting is usually the better route.

7. Under what circumstances would you consider Deep Learning?

- If we use the **raw transaction sequence** rather than aggregated behaviour. A per-customer time series of transactions is sequential data, where an LSTM or Transformer can learn patterns that flat aggregates lose.
- If we add **unstructured data** — bank statements as documents, call-centre transcripts, or scanned income proofs — since text/image handling is deep learning’s natural territory.
- If there are **very high-cardinality categorical features** where learned entity embeddings outperform one-hot or target encoding.
- If the problem becomes **relational/graph-shaped** — for example detecting coordinated default or fraud rings across linked accounts — where a Graph Neural Network is the right tool.

Absent one of those, deep learning adds cost, latency and explainability problems without a reliable accuracy gain.

Business implications: the model must produce adverse-action reasons for declined applicants, pass fairness testing on protected attributes (and on proxies for them), be documented

under model-risk governance, and be monitored for population drift — a credit model silently degrades as economic conditions change.

Scenario 2 — Finance: Financial Document Assistant

Thousands of annual reports, financial statements, policy, regulatory and investment documents. An analyst asks: “What are the major risks mentioned in the company’s latest annual report?”

1. Is this primarily an ML, DL or GenAI problem?

GenAI. Nothing is being predicted or classified — the analyst wants a natural-language answer synthesised from documents. That is a retrieval-plus-generation task. (Deep learning is of course inside the LLM and the embedding model, but as an application pattern this is GenAI, not a supervised learning problem.)

2. Would you use RAG or fine-tuning?

RAG.

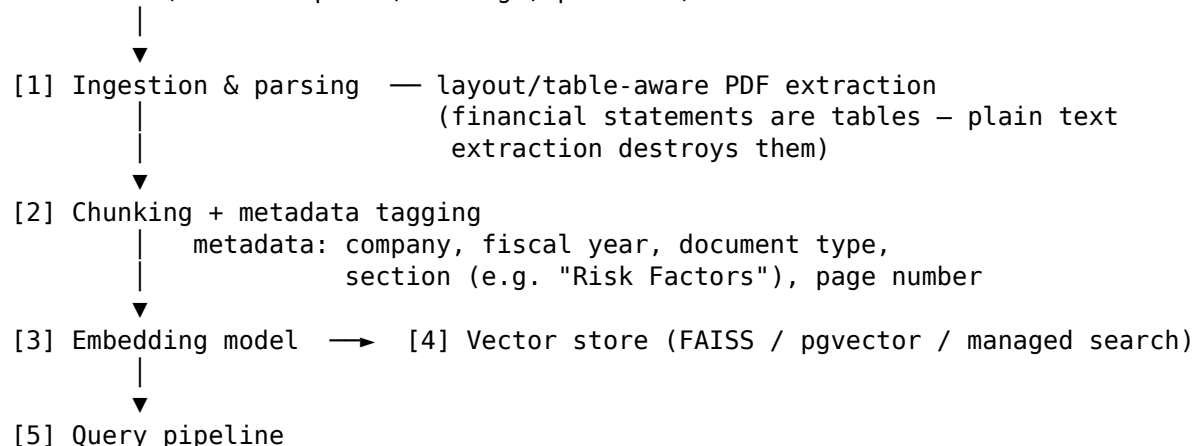
3. Why?

- **The knowledge changes constantly.** New annual reports and regulatory filings arrive continuously. With RAG you add a document to the index and it is immediately answerable; with fine-tuning you would have to retrain every reporting cycle.
- **Citations are non-negotiable.** An analyst must be able to verify a risk statement against the actual filing, and compliance needs an audit trail. RAG returns the source passage and page; fine-tuning bakes facts into weights with no traceability.
- **Fine-tuning teaches style, not facts.** It is good at making a model adopt a format or tone, and unreliable as a way to install factual knowledge — a fine-tuned model will still hallucinate confidently, and you cannot tell which document a statement came from.
- **Scope control.** “The company’s *latest* annual report” requires filtering to one company and one fiscal year. That is a metadata filter in retrieval — straightforward in RAG, essentially impossible to guarantee in a fine-tuned model, which may blend facts across companies and years.
- **Cost.** Indexing thousands of documents is cheap relative to repeated fine-tuning runs.

Fine-tuning could still play a small complementary role later — for example to enforce the house output format — but the factual content should always come from retrieval.

4. Design the architecture.

Documents (annual reports, filings, policies)



- └ query understanding: resolve "latest" → max(fiscal_year) for that company
- └ metadata filter: company = X AND fiscal_year = latest
- └ hybrid search: dense (semantic) + BM25 (exact terms, tickers, line items)
- └ cross-encoder reranker → top-k most relevant passages
- └ retrieval confidence check → refuse if below threshold

▼
[6] Grounded prompt + LLM — "answer only from context; cite page; say if absent"

▼
[7] Post-generation verification — every claim traceable to a retrieved passage?

▼
[8] Answer + citations → Analyst UI (+ audit log of query/answer/sources)

Two details matter for financial documents specifically. **Table-aware parsing** is essential, because naive text extraction turns a balance sheet into unusable number soup. And **metadata filtering is a correctness feature, not an optimisation** — without it, a question about one company's risks can retrieve another company's risk factors, which is worse than no answer.

5. How would you reduce hallucination?

- A strict grounding prompt with an explicit refusal instruction, and a fixed "not found in the provided documents" response.
- Mandatory **page-level citations** for every claim, so unsupported statements are visible immediately.
- **Metadata filtering** to prevent cross-company and cross-year contamination.
- **Reranking** so the context actually contains the answer — most hallucination is really a retrieval failure, where the model was handed irrelevant context and filled the gap itself.
- A **retrieval confidence threshold** — if the best match scores poorly, refuse rather than answer from a weak context.
- **Extractive answers for figures.** For any number, quote it from the source rather than letting the model restate it, since restated numbers are where subtle errors appear.
- A **verification pass** that checks each generated claim is entailed by the retrieved context.
- **Human review** for material figures used in investment decisions.

6. How would you evaluate retrieval quality?

Build a labelled evaluation set: a few hundred analyst questions, each mapped to the passages that genuinely contain the answer. Then measure:

- **Recall@k** — is the correct passage in the top k at all? This is the single most important retrieval metric, because if the answer is not retrieved, no amount of prompting will save the answer.
- **Precision@k / MRR / nDCG** — how highly ranked and how clean the retrieved set is.
- **Context precision and context recall** (RAGAS-style) for the end-to-end pipeline.
- **Faithfulness / groundedness** — the share of generated claims supported by the retrieved context.
- **Citation accuracy** — does the cited page actually support the sentence it is attached to?
- **Refusal rate on out-of-scope questions** — deliberately ask things the corpus cannot answer and confirm the system declines instead of inventing.

Scenario 3 — Telecom: Customer Churn

Predict which customers are likely to leave within the next 30 days, from call duration, data consumption, complaints, recharge frequency, plan type, tenure, network quality and service

interactions.

1. What type of ML problem is this?

Supervised binary classification with a fixed 30-day prediction horizon. It can alternatively be framed as **survival analysis** (time-to-churn), which is useful when the business wants to know *when* a customer is likely to leave rather than just whether they will within 30 days.

2. What would be the target variable?

churn = 1 if the customer churns within 30 days of the prediction date, else 0. “Churn” needs a concrete operational definition — for prepaid, typically no recharge or no revenue activity for a defined period (e.g. 30 days of inactivity, or port-out confirmed); for postpaid, a disconnection request or contract non-renewal.

The critical design point is the **snapshot construction**: all features must be computed strictly from data *before* the prediction date, and the label from the 30-day window *after* it. Mixing the two — for example including “complaints in the last 30 days” when those complaints happened during the label window — creates target leakage and produces a model that looks excellent offline and fails in production.

3. Which models would you consider?

Logistic Regression as an interpretable baseline; Random Forest as a non-linear benchmark; **XGBoost / LightGBM** as the likely best performer on this kind of tabular behavioural data. If the business also wants timing, a **Cox proportional-hazards** or gradient-boosted survival model. If usage shows strong sequential patterns (a steady decline in data consumption over weeks), a sequence model on the raw usage time series is worth testing — but only after the tabular baseline is established.

4. Which evaluation metric would you prioritise?

Lift / recall in the top deciles, and **PR-AUC** overall. The reason is operational: the retention team can only contact a limited number of customers, so what matters is not global accuracy but “among the top 5,000 customers we flag, how many would actually have churned?” That is precision@k and lift@decile. PR-AUC captures overall ranking quality under imbalance; ROC-AUC is worth reporting but is less sensitive to what matters here.

5. How would you convert model predictions into business actions?

- **Score and rank weekly**, then prioritise by churn probability × customer value (ARPU or CLV) — saving a high-value customer at moderate risk is worth more than a low-value customer at high risk.
- **Explain each prediction** with SHAP to get the churn *driver* per customer, then match the intervention to the driver: poor network quality → raise a network ticket and inform the customer of the fix; price sensitivity → targeted discount or better-fit plan; repeated complaints → proactive service recovery call. Sending a discount to someone leaving because of coverage wastes margin and does not retain them.
- **Set thresholds by ROI**, not by 0.5: intervene only where the offer cost is less than the expected retained margin.
- **Measure with a holdout**. Always keep a randomised control group so incremental retention can be measured rather than assumed.
- **Prefer uplift modelling** as the next step. A churn model tells you who will leave; an uplift model tells you who will *change behaviour because you contacted them*, which avoids spending on customers who would have stayed anyway (or who are already lost).

6. How could GenAI complement the churn model?

- Turn SHAP output into **plain-language explanations** for retention agents (“this customer’s data usage dropped 60% and they logged two network complaints in the last

month”).

- **Generate personalised retention messages and offers** matched to each customer’s churn driver, at scale.
- **Summarise the customer’s history** (complaints, calls, prior offers) into a briefing so the agent starts the conversation informed.
- **Mine unstructured text** — complaint tickets and call transcripts — into structured features (sentiment, topic, escalation) that feed the churn model itself, which is often where the largest accuracy gain comes from.
- Act as an **agent copilot** recommending the next-best action and drafting the follow-up.

Note the division of labour: the ML model does the prediction, GenAI does the language and the last-mile personalisation. Using an LLM to do the churn prediction itself would be the wrong tool.

Scenario 4 — Telecom: Network Operations Copilot

Incident logs, troubleshooting manuals, SOPs, equipment documentation and engineer notes. An engineer asks: “Cell site XYZ has experienced repeated packet-loss incidents. What troubleshooting steps should I follow?”

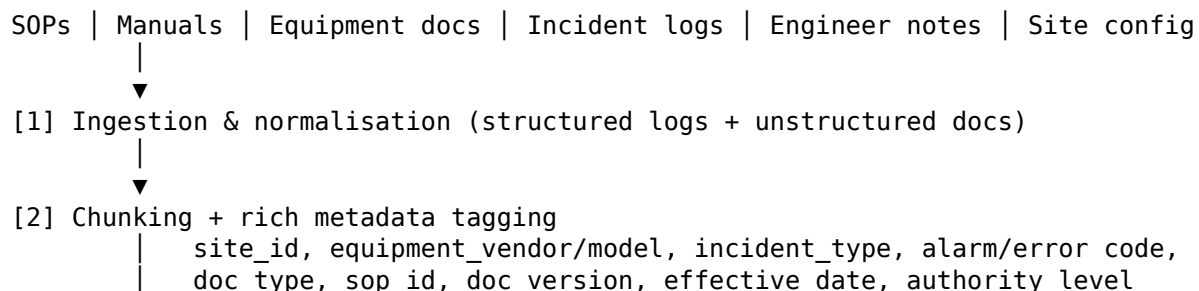
1. Why would RAG be appropriate?

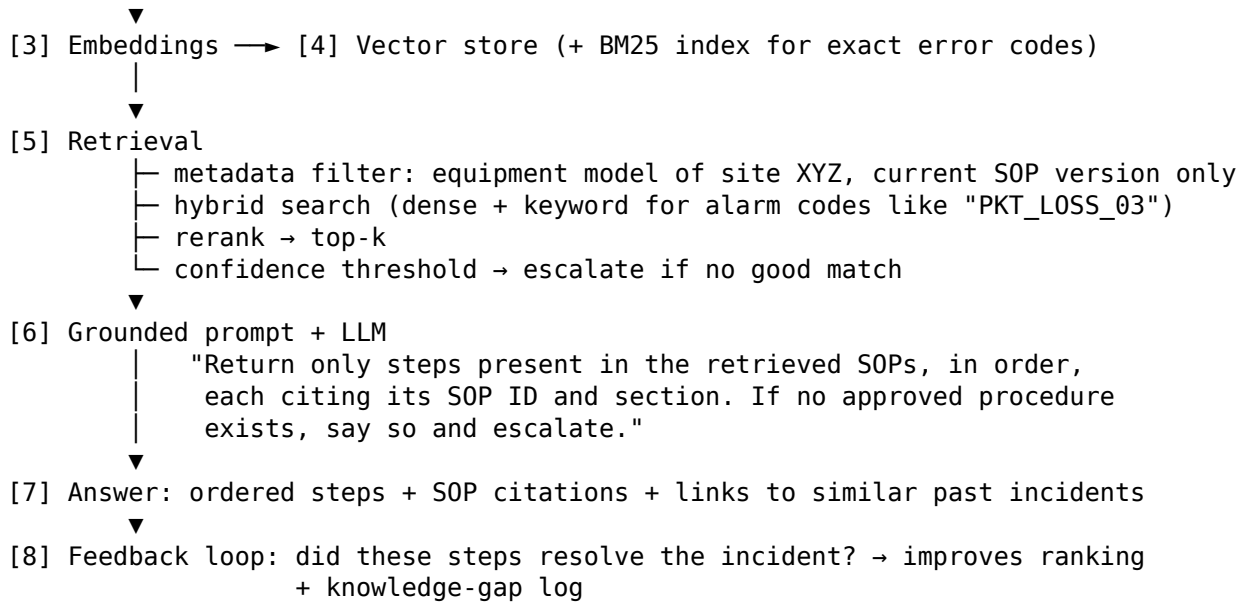
Because the answer must be the **company’s own approved procedure**, not a plausible-sounding general one. A generic LLM might produce reasonable-sounding network troubleshooting advice that does not match this operator’s equipment, escalation policy or safety rules — and acting on invented steps on live network equipment can cause an outage. RAG grounds every answer in the actual SOP and returns the source, so the engineer can verify it. The knowledge base also changes constantly as equipment and procedures are updated, which suits re-indexing far better than retraining, and the historical incident log is a large, continuously growing corpus that is valuable precisely because it is specific and current.

2. What information should be indexed?

- **SOPs and troubleshooting manuals** — the core approved procedures.
- **Equipment documentation** per vendor and model, since steps differ by hardware.
- **Historical incident logs with their resolutions** — often the highest-value source, because “what fixed this symptom last time on this equipment” is exactly what the engineer wants.
- **Engineer notes / field observations** — tacit knowledge not in formal manuals (tagged as lower authority than an approved SOP).
- **Site and network configuration metadata** — site XYZ’s equipment, topology, recent changes.
- **Change logs and maintenance records** — repeated packet loss right after a config change is a strong clue.

3. Design the RAG architecture.





An optional extension is an **agentic tool call** to pull live telemetry for site XYZ, so the copilot can combine the SOP with the site's current state — but the procedure itself must still come from retrieval.

4. How would you prevent the LLM from inventing troubleshooting procedures?

- **Extractive-first design.** The retrieved SOP steps are returned largely verbatim; the LLM's job is to select, order and present them, not to compose new ones. This is the single most effective control.
- **Mandatory citation per step** — every step carries its SOP ID and section, so an uncited step is immediately visible as suspect.
- **Explicit refusal instruction** with a fixed response when no approved procedure matches.
- **Retrieval confidence threshold** — a weak best-match triggers escalation instead of an answer.
- **Version filtering** so only the current approved SOP can be retrieved, preventing a retired procedure from being presented as current.
- **Read-only scope** — the copilot recommends, it does not execute changes on network equipment. High-risk actions require human approval.
- **Authority ranking** in metadata, so an approved SOP outranks an informal engineer note, and the note is labelled as such when used.

5. What metadata could improve retrieval?

site_id, region, equipment_vendor, equipment_model, firmware_version, incident_type, alarm/error_code, severity, sop_id, doc_version, effective_date/superseded_by, author_team, authority_level, and resolution_success_rate from the incident history. These let retrieval narrow to the right hardware and the currently valid procedure — filtering by equipment model and current version typically improves usable precision more than any embedding upgrade, because it removes whole classes of confidently-wrong-but-semantically-similar matches.

6. What would happen if the relevant procedure does not exist in the knowledge base?

The system must **say so explicitly and escalate — never improvise**. Concretely:

1. Return a clear message: *“No approved troubleshooting procedure exists in the knowledge base for repeated packet loss on this equipment type.”*
2. Show the **closest related documents** it did find, clearly labelled as related-but-not-matching, so the engineer has a starting point without being misled.
3. Surface any **similar historical incidents** and how they were resolved, labelled as precedent rather than approved procedure.
4. **Route to escalation** (L2/L3) and optionally auto-create a ticket with the retrieval trace attached.
5. **Log the gap** into a knowledge-gap report, so the documentation team can author the missing SOP. Over time this turns the copilot into a tool that actively improves the knowledge base.

This behaviour is a feature, not a limitation: in network operations, an honest “I don’t have an approved procedure for this” is far more valuable than a fluent invention.

Scenario 5 — Healthcare: Patient Readmission

Predict whether a patient will be readmitted within 30 days, from age, diagnosis, length of stay, previous admissions, medication count, lab results and comorbidities.

1. Would you use ML or DL?

Machine Learning — gradient boosting, with regularised Logistic Regression as the interpretable benchmark. The features are structured clinical variables, dataset sizes are typically in the tens of thousands rather than millions, and clinical deployment demands explainability and auditability that tree models with SHAP provide far more readily than a neural network. Deep learning becomes relevant only if we extend the input to **unstructured discharge summaries and clinical notes** (NLP), or to **time-series vitals** from monitoring — both genuinely deep-learning-shaped, and both usually a second phase.

2. What would the target variable be?

`readmitted_30d = 1` if the patient has an **unplanned** readmission within 30 days of discharge from the index admission. The definition needs care: exclude *planned* readmissions (scheduled chemotherapy or staged surgery are not failures of care), exclude patients who died in hospital or were transferred, define the index admission clearly, and handle patients readmitted to a different facility if that data is available. It is also worth aligning the definition to the relevant regulatory/payer measure so the model’s output is comparable to the metric the hospital is actually judged on.

3. Which evaluation metrics would you consider?

- **Recall / sensitivity** — the primary metric, for the reason in Q4.
- **PR-AUC** — readmissions are the minority class, so this is more informative than ROC-AUC.
- **Precision at the alert volume the care team can handle** — a model flagging 40% of discharges is not deployable regardless of its recall.
- **Calibration** (calibration curve, Brier score) — especially important clinically. Clinicians reason with probabilities, so “this patient is at 30% risk” must actually mean roughly 30%. A poorly calibrated model erodes trust quickly.
- **Decision-curve analysis / net benefit** — evaluates the model at clinically realistic thresholds rather than in the abstract.

4. Why could false negatives be particularly important?

A false negative is a patient who *will* be readmitted but is scored low-risk. They are therefore not enrolled in transitional care — no follow-up call, no medication reconciliation, no early out-

patient appointment. The consequence is direct patient harm: an avoidable deterioration and an avoidable hospitalisation, which for a frail patient carries real risk of complications. There is also a financial dimension, since avoidable readmissions attract penalties under schemes such as HRRP.

By contrast, a false positive means a patient receives a follow-up call and a care-management review they did not strictly need — low cost and essentially harmless. With that asymmetry, the model should be tuned for high sensitivity, with the threshold set so the number of flagged patients stays within the care team’s actual capacity.

5. How would you explain the prediction to a healthcare professional?

- Give the **risk in absolute terms alongside the base rate** (“28% predicted risk, against a 12% average for similar admissions”) rather than a bare score or an unexplained “high risk” label.
- Provide **per-patient SHAP contributions in clinical language**: “risk raised by 3 admissions in the past 6 months, 12 active medications and heart-failure comorbidity; lowered by short length of stay and normal renal function.”
- Show **global feature importance** separately, so the clinician understands how the model behaves in general and can sanity-check it against clinical knowledge.
- Frame it explicitly as **decision support, not a diagnosis** — the model estimates risk, it does not decide care.
- Make it **actionable**: pair the risk with the intervention it should trigger, since a score with no recommended action gets ignored.
- State **limitations plainly** — the population it was trained on, and where it is likely to be unreliable.

6. What risks should be considered before deploying the model?

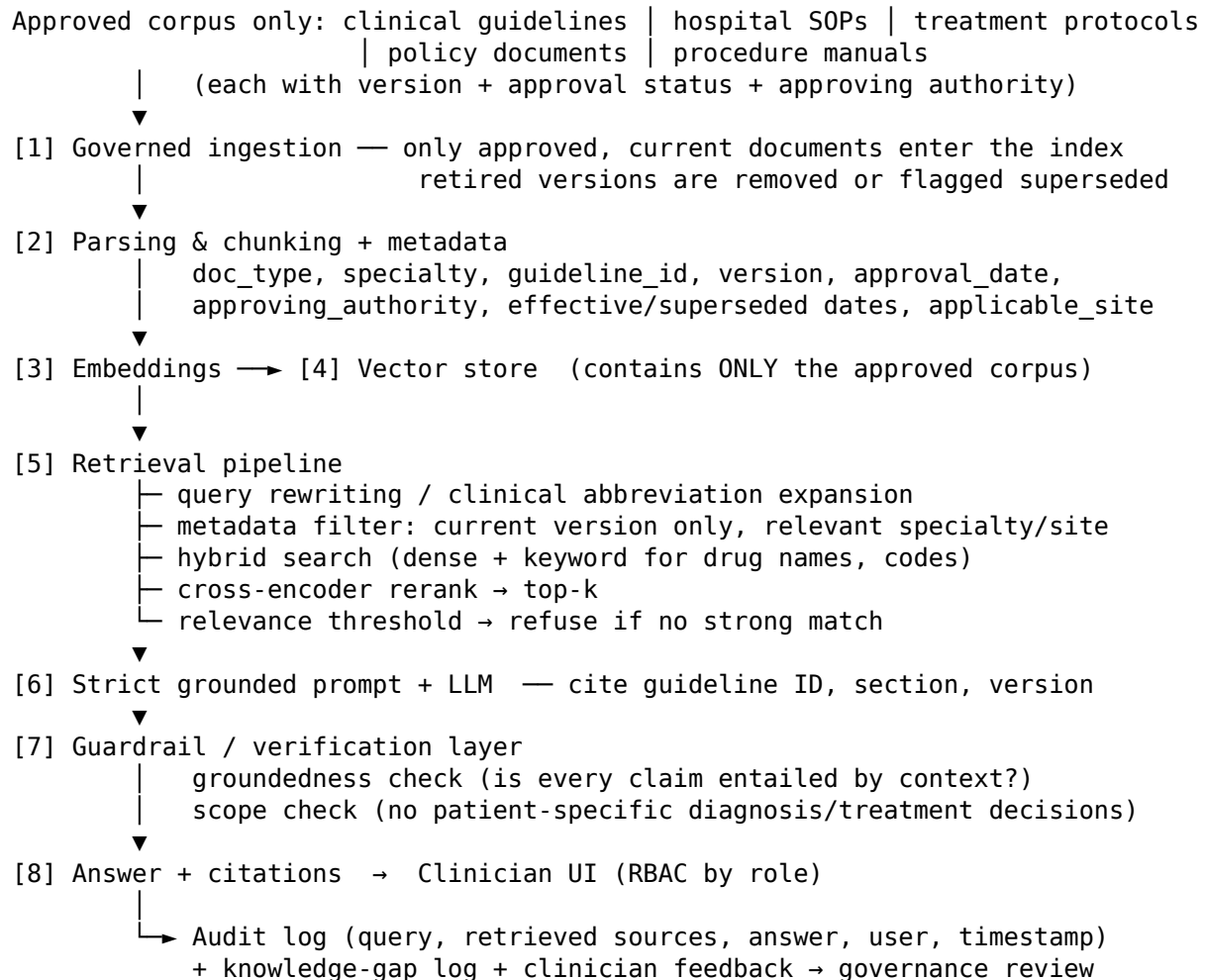
- **Bias and health equity.** This is the most serious risk. If the label or features encode historical inequities in access to care, the model can systematically under-flag disadvantaged groups. There is a well-known real-world case where using healthcare *cost* as a proxy for health *need* substantially under-identified Black patients for care programmes. Subgroup performance auditing before deployment is mandatory, not optional.
- **Target leakage** — any feature recorded after discharge (or derived from the readmission itself) will inflate offline performance and fail in production.
- **External validity / drift** — a model trained elsewhere, or on pre-pandemic data, may not transfer. Validate locally and monitor for drift as coding practices and care pathways change.
- **Alert fatigue** — too many flags and clinicians stop responding, which destroys the intervention’s effect regardless of model quality.
- **Automation bias** — clinicians deferring to a score against their own judgement. The workflow must make override easy and expected.
- **Privacy and governance** — PHI handling, access control, and a documented model-governance process with a named owner.
- **Regulatory classification** — whether the tool counts as clinical decision support or a regulated medical device in the relevant jurisdiction, which changes the compliance burden.
- **A monitoring and rollback plan**, with human oversight and clear accountability if the model is wrong.

Scenario 6 — Healthcare: Clinical Knowledge Assistant

An assistant that answers questions using only approved clinical guidelines, hospital SOPs, treatment protocols, policy documents and procedure manuals. “The assistant must not an-

swer using information outside our approved documents.”

1. Design the architecture.



2. Why is RAG appropriate?

The organisation’s requirement — “*must not answer using information outside our approved documents*” — is essentially a description of RAG. RAG makes the approved corpus the only knowledge source consulted at answer time, so scope compliance is enforced by architecture rather than by hoping the model behaves. It also fits three other realities of clinical practice: guidelines are **updated frequently** (re-index rather than retrain), answers must be **citable** for verification and medico-legal defensibility, and a retired guideline must be **removable immediately** — with RAG you delete it from the index and it can no longer be cited, which is impossible once knowledge is baked into model weights.

3. What would the retrieval pipeline look like?

Query → normalise and expand clinical abbreviations (a query for “MI” should match “myocardial infarction”) → embed with the same model used for the corpus → **metadata-filtered** vector search restricted to the approved corpus, current versions, and relevant specialty/site → hybrid keyword search in parallel to catch exact drug names, dosages and codes → merge candidates → **cross-encoder rerank** → apply a **relevance-score threshold** → if nothing passes, refuse and escalate; if it passes, assemble the context with citation metadata attached → grounded prompt → LLM → post-generation groundedness verification → return answer with citations, or refuse.

4. How would you handle questions for which no answer exists in the knowledge base?

Refuse explicitly and route to a human. The response should state plainly that the question is not covered by the approved documents, list the closest approved documents that *were* searched (clearly labelled as related but not answering the question), and direct the clinician to the appropriate escalation path — clinical informatics, the relevant specialist, or the guideline owner. The query is logged as a **knowledge gap** so the guideline committee can decide whether to author new guidance.

What must *not* happen is a silent fallback to the base model’s general medical knowledge. That fallback is the single biggest risk in a system like this, because the answer will sound authoritative and indistinguishable from a grounded one. Blocking it requires all three of: an explicit prompt instruction, a retrieval threshold, and an independent groundedness check that fails closed.

5. How would you evaluate hallucination?

- Build a **gold evaluation set** of clinical questions with known correct answers and known source passages, curated with clinicians.
- Build a deliberate **out-of-scope / unanswerable set** and measure the **refusal rate** — a system that answers these is hallucinating by construction.
- Measure **faithfulness / groundedness**: decompose each answer into individual claims and check each is entailed by the retrieved context (automated entailment checking, sampled and confirmed by clinician review).
- Measure **citation accuracy** — does the cited guideline section actually support the sentence attached to it? A correct answer with a wrong citation is still a failure.
- **Red-team** with leading questions (“confirm that drug X is first-line for Y”), questions premised on falsehoods, and prompt-injection text embedded in ingested documents.
- Track hallucination rate as a **monitored production metric**, with ongoing sampled clinician audit rather than a one-off pre-launch test.

6. What safeguards would you implement?

- **Grounding + mandatory citations**, with a hard refusal path and no general-knowledge fallback.
- **Approved-corpus-only index** with version control and governed ingestion.
- **Retrieval confidence thresholds** and an independent groundedness verification step that fails closed.
- **Explicit scope limits** — the assistant answers questions *about the guidelines*; it does not make diagnostic or treatment decisions for an individual patient.
- **Clinician-in-the-loop** for anything patient-specific, and easy override.
- **RBAC by role** and **full audit logging** of every question, retrieved source and answer.
- **PHI/PII handling** — de-identification, data-residency and retention controls.
- **Prompt-injection defences** on ingested documents (document text is data, never instructions).
- **Change control and clinical governance sign-off** before any guideline or prompt change goes live.
- **Clear non-emergency positioning** — emergencies must route to humans and emergency protocols, not to an assistant.
- **Monitoring, incident response and rollback**, with a named accountable owner.

7. Would you fine-tune the LLM? Explain your reasoning.

No — not for the clinical knowledge itself. Fine-tuning would work directly against every requirement in this scenario:

- It **cannot be cited**. Knowledge in the weights has no source, so an answer cannot be

traced to an approved guideline — which breaks both clinical verification and medico-legal defensibility.

- It **cannot be updated or retracted cleanly**. When a guideline is superseded, RAG drops it from the index instantly. A fine-tuned model has effectively memorised the old guidance and will keep reproducing it until retrained, with no guarantee the old fact is gone.
- It **increases hallucination risk of the most dangerous kind** — confident, fluent recall of possibly outdated clinical guidance, indistinguishable in tone from correct guidance.
- It **breaks the stated constraint**. Once trained on this content, the model blends it with its pre-existing general medical knowledge, and you can no longer demonstrate that an answer came only from approved documents.
- It is **more expensive and slower to maintain** than re-indexing.

The **legitimate exception is format, not facts**. Light fine-tuning could be justified to make the model reliably follow the hospital’s answer template, tone, or structured output — while all clinical content still comes from retrieval. Even then I would exhaust prompt engineering and few-shot examples first, and only fine-tune if format compliance measurably fails at scale. The rule of thumb: **RAG for what the model should know, fine-tuning for how it should speak**.

Part D — Summary: choosing between ML, DL, GenAI and RAG

Signal in the problem	Right choice
Predict a label/number from structured tabular data	ML (gradient boosting; logistic regression as interpretable baseline)
Sequential, unstructured or very high-dimensional input (transaction sequences, images, notes, audio); or graph structure	DL
Generate, summarise or converse in natural language	GenAI
Answers must come from a specific, changing, citable document corpus	RAG
Need the model to adopt a house style or output format	Fine-tuning (or better, prompt engineering first)

Two cross-cutting lessons run through all six scenarios. First, **the metric must match the cost of the error** — recall dominates when a miss is expensive (credit default, patient readmission), while precision@k dominates when the intervention budget is capped (churn outreach). Second, for every knowledge-retrieval system, **the ability to refuse is a core feature**: in network operations and clinical guidance alike, an honest “there is no approved answer for this in the knowledge base” is materially more valuable than a fluent invention.